

INE5622 - Introdução a Compiladores

Analizador Léxico e Sintático

Entrega: 26 de junho de 2026 (até 23:55h via Moodle)

Este Exercício-Programa (EP) pode ser realizado por grupos (com até 5 integrantes). Cada grupo deverá executar as seguintes tarefas:

- Construção de um analisador léxico para uma linguagem (**AL**); e
- Construção de um analisador sintático para uma linguagem (**AS**).

A avaliação será realizada através da análise léxica e sintática para algumas entradas.

Trabalharemos com uma linguagem denominada LCC-2026-1. Os *tokens* (os terminais da gramática) associados a essa linguagem estão disponível no fim deste texto. Se desejarem, os grupos poderão realizar *pequenas* modificações na linguagem. No entanto, qualquer modificação deverá ser detalhada no cabeçalho dos arquivos que definem os analisadores léxico e sintático

A nota deste EP é $T = T_1 + T_2 + T_3$, onde T_1 está definida na seção 4, T_2 está definida na seção 5 e T_3 está definida na seção 6.

1 Tarefa AL

A tarefa **AL** consiste na implementação de um analisador léxico para uma linguagem de programação. O analisador léxico precisa necessariamente ler caracter por caracter da entrada e deve ser baseado em diagramas de transição. Uma *tabela de símbolos* deverá ser implementada. Essa tabela deverá ter uma entrada por *token* **Identificador** e deverá manter uma lista de ocorrências do identificador no arquivo (linha e coluna onde ocorre este token).

Todos os integrantes dos grupos devem dominar qualquer questão relacionada à tarefa **AL**.

2 Tarefa AS

A tarefa **AS** consiste na implementação de um analisador sintático para uma linguagem de programação. A gramática associada (**CC-2026-1**) **deve** estar em LL(1). O analisador sintático deve construir uma tabela de reconhecimento sintático uma única vez. Depois disso, o analisador deverá usar essa tabela para demonstrar (ou não) a pertinência da entrada na linguagem gerada pela gramática (LCC-2026-1). Abaixo há algumas sugestões que podem ajudar a alcançar tal objetivo.

1. CC-2026-1 está na forma BNF. Coloque-a na forma *convencional* de gramática. Essa gramática na forma convencional será a LCC-2026-1.
2. Remova recursão à esquerda de LCC-2026-1, se existir.
3. Fatore LCC-2026-1 à esquerda, se ela não estiver fatorada.
4. Faça LCC-2026-1 ser uma gramática em LL(1). É permitido adicionar novos terminais na gramática, caso necessário. Utilize o teorema visto em aula para demonstrar que a gramática resultante está em LL(1).

5. Depois que LCC-2026-1 estiver em LL(1), construa a tabela de reconhecimento sintático.

Mais uma vez, todos os integrantes dos grupos devem dominar qualquer questão relacionada à tarefa **AS**.

3 O que deve ser entregue?

A data para entregar o EP é dia 26 de junho de 2026 (até 23:55h via Moodle). Cada grupo deverá entregar um conjunto de arquivos com:

1. um conjunto de arquivos que definem os analisadores léxico e sintático (pode ser um único arquivo ou vários arquivos);
2. três programas escritos na linguagem LCC-2026-1 (com pelo menos 100 linhas cada, sem erros léxicos e sem erros sintáticos);
3. um *Makefile* para compilação/interpretação/execução do analisador léxico e sintático;
4. um README com informações importantes para a execução apropriada de todos os programas desenvolvidos.

Orientações para construção de um *Makefile*: <https://www.gnu.org/software/make/manual/make.html>

4 O que será avaliado na análise léxica?

Chamamos de T_1 a nota para a avaliação da análise léxica. $0 \leq T_1 \leq 3,0$. Será observada a construção da tabela de símbolos e a execução do analisador léxico lendo caracter por caracter da entrada e baseado em diagramas de transição. Serão inseridos erros léxicos na entrada que deverão ser capturados pelo analisador léxico.

5 O que será avaliado na análise sintática?

Chamamos de T_2 a nota para a avaliação da execução da análise sintática. $0 \leq T_2 \leq 4,0$. Será observada a construção da tabela de reconhecimento sintático (uma única vez), e a execução do analisador sintático para LCC-2026-1 em LL(1). Serão inseridos erros sintáticos na entrada que deverão ser capturados pelo analisador sintático.

6 Sobre as execuções dos programas desenvolvidos

Será medido o tempo de compilação de cada programa desenvolvido (ou seja, será realizada uma média do tempo de execução da análise léxica e sintática para entradas sem erros léxicos e sintáticos). O grupo que tiver a média do tempo de compilação mais rápido terá $T_3 = 3,0$. Seja t tal tempo. Para os demais grupos, $T_3 = 3,0 \times \frac{t}{t_g}$, onde t_g é o tempo do outro grupo ($t_g \geq t$).

No momento da execução dos programas desenvolvidos por um grupo, a presença de seus integrantes poderá ser necessária para a efetiva avaliação.

7 Outros itens importantes que devem ser considerados

Abaixo listamos itens importantes com relação a essa avaliação.

- A existência de três programas para LCC-2026-1 com extensão `.lcc` e com pelo menos 100 linhas cada, sem erros léxicos e sem erros sintáticos (se não existir os três nas condições citadas, então $T = 0$);
- A existência de um *Makefile* (se algum não existir, então $T = 0$);
- A execução correta do *Makefile* (se algum não executar corretamente, então $T = 0$);
- A existência de um README (se não existir, então $T = 0$);
- A compilação dos programas desenvolvidos (se houver erros de compilação/ interpretação, então haverá descontos em T);

8 Sobre a entrada e a saída dos dados

Para cada execução, uma única entrada será dada: o caminho de um arquivo no formato `lcc` escrito na linguagem LCC-2026-1.

Exemplo de uma entrada: `/tmp/arvore-binaria-de-busca.lcc`.

As seguintes saídas são esperadas:

- para o analisador léxico:
 - se não houver erros léxicos $\rightarrow$ uma lista de *tokens* (na mesma ordem em que eles ocorrem no arquivo dado na entrada) e uma tabela de símbolos;
 - se houver erros léxicos $\rightarrow$ uma mensagem simples de erro léxico indicando a linha e a coluna do arquivo de entrada onde ele ocorre.
- para o analisador sintático:
 - se não houver erros sintáticos $\rightarrow$ uma mensagem de sucesso;
 - se houver erros sintáticos $\rightarrow$ uma mensagem de insucesso indicando qual é a entrada na tabela de reconhecimento sintático que está vazia (qual é a forma sentencial α , qual é o símbolo não-terminal mais à esquerda de α e qual é o *token* da entrada).

Observações importantes:

1. Os programas podem ser escritos em C (compatível com compilador gcc versão 12.2.0), C++ (compatível com compilador g++ versão 12.2.0), Java (compatível com compilador javac versão 17.0.16), Python 3 (compatível com versão 3.11.2) ou Rust (compatível com rustc versão 1.88.0) e deve ser compatível com Linux/Unix.
2. Se for desenvolver em Python 3, então especifique (no *Makefile* principalmente) qual é a versão que está usando. Prepare seu *Makefile* considerando a versão usada.
3. Exercícios-Programas atrasados **não** serão aceitos.
4. Programas com *warning* na compilação terão diminuição da nota.
5. É importante que seu programa esteja escrito de maneira a destacar a estrutura do programa.
6. O relatório e o programa devem começar com um cabeçalho contendo pelo menos o nome de todos os integrantes do grupo.
7. Coloque comentários em pontos convenientes do programa, e faça uma saída clara.
8. A entrega do Exercício-Programa deverá ser feita no Moodle.
9. O Exercício-Programa é individual por grupo. Não copie o programa de outro grupo, não empreste o seu programa para outro grupo, e tome cuidado para que não copiem seu programa sem a sua permissão. Todos os programas envolvidos em cópias terão nota *T* igual a ZERO.

Bom trabalho!

Na próxima página você encontrará uma gramática CC-2026-1 na forma BNF. Assim como as expressões regulares, as gramáticas também descrevem linguagens. No entanto, as gramáticas são mais poderosas pois podem descrever linguagens que expressões regulares não podem. A gramática abaixo é fortemente baseada na gramática X++ de Delamaro. Para o trabalho relacionado ao analisador léxico, precisamos destacar os *tokens*. Eles são precisamente os *símbolos terminais* de CC-2026-1 e que estão na cor amarela. Os símbolos terminais não-triviais são somente *ident*, *int_constant*, *float_constant* e *string_constant*. Os *símbolos não-terminais* de CC-2026-1 estão em letra de forma. Os demais símbolos (na cor azul) são símbolos da notação BNF. Consulte o livro de Delamaro para mais informações sobre a notação BNF (seção 2.3 - página 12).

Livro do Delamaro: <http://conteudo.icmc.usp.br/pessoas/delamaro/SlidesCompiladores/CompiladoresFinal.pdf>

Em seguida apresentamos uma entrada (um código fonte) da linguagem descrita pela gramática LCC-2026-1.

```
def func1(int A, int B)
{
    int SM[2];
    SM[0] = A + B;
    SM[1] = B * C;
    return;
}

def principal()
{
    int C;
    int D;
    int R;
    C = 4;
    D = 5;
    R = func1(C, D);
    return;
}
```

Para este exemplo, a lista de tokens é [def, ident, (, int, ident, ,, int, ident,), {, int, ident, ..., return, ;, }]. A lista de tokens que deve ser elaborada pelo seu analisador léxico deverá ser similar a essa. A tabela de símbolos deverá conter uma entrada para cada token *ident* com pelo menos um atributo (por exemplo, a linha onde ocorre tal token – SM ocorre nas linhas 3, 4 e 5).

<i>PROGRAM</i>	→ (<i>STATEMENT</i> <i>FUNCLIST</i>)?
<i>FUNCLIST</i>	→ <i>FUNCDEF</i> <i>FUNCLIST</i> <i>FUNCDEF</i>
<i>FUNCDEF</i>	→ <i>def ident</i> (<i>PARAMLIST</i>){ <i>STATELIST</i> }
<i>PARAMLIST</i>	→ ((<i>int</i> <i>float</i> <i>string</i>) <i>ident</i> , <i>PARAMLIST</i> (<i>int</i> <i>float</i> <i>string</i>) <i>ident</i>)?
<i>STATEMENT</i>	→ (<i>VARDECL</i> ; <i>ATRIBSTAT</i> ; <i>PRINTSTAT</i> ; <i>READSTAT</i> ; <i>RETURNSTAT</i> ; <i>IFSTAT</i> <i>FORSTAT</i> { <i>STATELIST</i> } <i>break</i> ; ;)
<i>VARDECL</i>	→ (<i>int</i> <i>float</i> <i>string</i>) <i>ident</i> ([<i>int_constant</i>])*
<i>ATRIBSTAT</i>	→ <i>LVALUE</i> = (<i>EXPRESSION</i> <i>ALLOCEXPRESSION</i> <i>FUNCCALL</i>)
<i>FUNCCALL</i>	→ <i>ident</i> (<i>PARAMLISTCALL</i>)
<i>PARAMLISTCALL</i>	→ (<i>ident</i> , <i>PARAMLISTCALL</i> <i>ident</i>)?
<i>PRINTSTAT</i>	→ <i>print</i> <i>EXPRESSION</i>
<i>READSTAT</i>	→ <i>read</i> <i>LVALUE</i>
<i>RETURNSTAT</i>	→ <i>return</i>
<i>IFSTAT</i>	→ <i>if</i> (<i>EXPRESSION</i>) <i>STATEMENT</i> (<i>else</i> <i>STATEMENT</i>)?
<i>FORSTAT</i>	→ <i>for</i> (<i>ATRIBSTAT</i> ; <i>EXPRESSION</i> ; <i>ATRIBSTAT</i>) <i>STATEMENT</i>
<i>STATELIST</i>	→ <i>STATEMENT</i> (<i>STATELIST</i>)?
<i>ALLOCEXPRESSION</i>	→ <i>new</i> (<i>int</i> <i>float</i> <i>string</i>) ([<i>NUMEXPRESSION</i>])+
<i>EXPRESSION</i>	→ <i>NUMEXPRESSION</i> ((< > <= >= == !=) <i>NUMEXPRESSION</i>)?
<i>NUMEXPRESSION</i>	→ <i>TERM</i> ((+ -) <i>TERM</i>)*
<i>TERM</i>	→ <i>UNARYEXPR</i> ((* / %) <i>UNARYEXPR</i>)*
<i>UNARYEXPR</i>	→ ((+ -)?) <i>FACTOR</i>
<i>FACTOR</i>	→ (<i>int_constant</i> <i>float_constant</i> <i>string_constant</i> <i>null</i> <i>LVALUE</i> (<i>NUMEXPRESSION</i>))
<i>LVALUE</i>	→ <i>ident</i> ([<i>NUMEXPRESSION</i>])*